Energy-Efficient 3D Point Cloud Classification with Spiking Neural Networks

Learnable LIF Neurons, KNN Backbone, and
Bidirectional Temporal Processing

Technical Report — Purdue Project

Purdue SNN-PointNet Research

March 13, 2026

Contents

1 Introduction

Three-dimensional point cloud classification is a fundamental task in robotics, autonomous driving, and augmented reality. Standard deep networks (ANNs) achieve high accuracy but require floating-point multiply-accumulate (MAC) operations that are energy-intensive on edge hardware.

Spiking Neural Networks (SNNs) offer a biologically-inspired alternative: neurons communicate via binary spikes (0 or 1) rather than real-valued activations, replacing costly MACs with cheap spike-driven accumulations (ACs) on neuromorphic hardware such as Intel Loihi 2. On such hardware, an AC costs approximately $E_{\text{AC}} = 2.3 \times 10^{-3}$ pJ versus $E_{\text{MAC}} = 8.4 \times 10^{-3}$ pJ for a MAC [3] — a 3.7× per-operation savings that compounds with the firing-rate sparsity of trained SNNs.

This report presents a complete SNN framework for point cloud classification with three novel contributions:

1. **Learnable per-neuron LIF** — trainable leak τ and threshold ϑ per neuron, enabling the model to optimise spike sparsity jointly with accuracy.
2. **KNN Local Backbone** — K-nearest-neighbour graph inside each temporal slice captures local geometry that global PointNet-style MLPs miss.
3. **Bidirectional Temporal SNN** — dual forward/backward LIF passes over the slice sequence, fused before classification to provide full temporal context.

We benchmark against four published SNN methods (E-3DSNN, SpikingSSMs, SPT, SPM) and three ANN baselines (PointNet, DGCNN, PCT) on ModelNet10 and ModelNet40.

2 Datasets

ModelNet10 4,899 CAD models across 10 everyday object categories (chair, table, bed, etc.), split 3,991 train / 908 test.

ModelNet40 12,311 models across 40 categories, split 9,843 train / 2,468 test.

Each object is represented as $N = 1{,}024$ points in (x, y, z) coordinates, sampled from mesh surfaces and normalised to the unit sphere.

ScanObjectNN 15,000 real-world LiDAR scans with real-world clutter and occlusion. We use the three standard splits: OBJ-BG (with background), OBJ-ONLY (object only), and PB-T50-RS (hardest, perturbed). (*Full results pending — download issues during Kaggle run.*)

3 Temporal Slicing

SNNs require a *temporal* input sequence. Since point clouds are static, we define a **temporal slicing** procedure that partitions the N points into $T = 16$ ordered slices of $N/T = 64$ points each.

3.1 Radial Slicing (Baseline)

Points are sorted by Euclidean distance from the cloud centroid and chunked sequentially. Slice t contains the 64 points at the t-th range shell. This creates a centre-to-surface temporal ordering but generates highly correlated adjacent slices.

3.2 FPS Slicing (Ours)

Farthest Point Sampling (FPS) is applied to select $M = 64$ spatially diverse anchor points. Remaining points are assigned to the nearest anchor via KNN, then each group is spread across the T timesteps. This ensures every slice contains spatially distributed points from across the whole object, preventing the "information starvation" of early radial slices that only see the object interior.

Algorithm 1 — Hierarchical FPS Slicing

Input: $P \in \mathbb{R}^{B\times N\times 3}$, slices T
Output: $\{S_t\}_{t=0}^{T-1}$, each $S_t \in \mathbb{R}^{B\times (N/T)\times 3}$

1. Select $M = N/T$ anchors via iterative FPS.
2. Assign each remaining point to its nearest anchor.
3. For each anchor group, distribute points round-robin to timesteps.
4. Yield slice S_t = points assigned to timestep t.

Algorithm 1: Hierarchical FPS temporal slicing.

4 Model Architecture

All models share a two-stage design: a **per-slice backbone** that extracts spatial features from each slice independently, and a **temporal head** that integrates evidence across slices using spiking membrane dynamics.

4.1 Backbone Options

PointNet Backbone (baseline) A shared-weight MLP with LIF activations and hidden dimensions $[128, 256, 512]$. Each point is processed independently; global pooling aggregates to a slice embedding $\mathbf{e}_t \in \mathbb{R}^{512}$.

Local KNN Backbone [Novel] ours Augments the PointNet backbone with a K-nearest-neighbour graph ($k = 16$) within each slice. Each point's embedding is computed from itself and its k neighbours, capturing local surface geometry (edges, corners, curvature). This is analogous to DGCNN's EdgeConv [2] but applied slice-by-slice in the SNN pipeline.

4.2 Neuron Types

Standard LIF (baseline) Fixed leak $\tau = 0.9$ and threshold $\vartheta = 1.0$. Membrane update:

$$u_t = \tau\, u_{t-1} + W x_t \tag{1}$$
$$s_t = \Theta(u_t - \vartheta) \tag{2}$$
$$u_t \leftarrow u_t \cdot (1 - s_t) \quad \text{(hard reset)} \tag{3}$$

Backpropagation through the non-differentiable spike uses a surrogate gradient (triangular window): $\frac{\partial s}{\partial u} \approx \frac{1}{(1+|u|)^2}$.

Learnable LIF [Novel] ours Both τ and ϑ are **per-neuron learnable parameters**:

$$\tau_i = \sigma(\alpha_i) \in (0, 1) \quad \text{(sigmoid, initialised at 0.9)} \tag{4}$$
$$\vartheta_i = \text{softplus}(\beta_i) > 0 \quad \text{(initialised at 1.0)} \tag{5}$$

where $\alpha_i, \beta_i \in \mathbb{R}^{d_{\text{out}}}$ are learned. The model therefore jointly optimises classification accuracy *and* per-neuron spike timing. Prior work (PLIF [10]) learns one global τ per layer; ours learns d_{out} independent values, giving $O(d)$ more parameters but substantially more expressiveness.

4.3 Temporal Heads

TemporalSNN (baseline) Two stacked LIF layers process the slice embedding $\mathbf{e}_t$ at each step, maintaining running membrane state across T slices. The membrane at $t = T$ is passed to a linear classifier.

Bidirectional Temporal SNN [Novel] ours Inspired by the Time-Flip strategy of SPM [7], we maintain two independent LIF pairs:

- **Forward pair**: processes slices $t = 0 \to T-1$ (causal)
- **Backward pair**: processes slices $t = T-1 \to 0$ (anti-causal)

After all slices are processed, the final membrane potentials are element-wise summed and passed to the classifier:

$$\hat{y} = \text{FC}\Big(\mathbf{m}_T^{\text{fwd}} + \mathbf{m}_T^{\text{bwd}}\Big) \tag{6}$$

This allows each slice embedding to benefit from both past context (preceding slices) and future context (subsequent slices), which is especially useful for objects whose parts span multiple slices (e.g., a chair's seat, legs, and backrest each appear in different slices).

4.4 Active Spiking Perception (ASP) — Adaptive Slice Selection [Novel] ours

Standard fixed-T SNN models always process all T slices regardless of input difficulty. We introduce **Active Spiking Perception (ASP)**, a model that *selects* which slice to observe next based on the current membrane state, and exits early once the model is sufficiently confident.

Slice Selection Policy (SSP). At each step t, the SSP scores all unvisited slices $s \in \{0, \ldots, T-1\}$ via a cross-attention between the current membrane state $\mathbf{m}_t$ and a geometric descriptor $\mathbf{g}_s$ for each slice:

$$\text{score}(s) = \frac{(\mathbf{W}_k \mathbf{m}_t)^\top (\mathbf{W}_q \mathbf{g}_s)}{\sqrt{d_{\text{ssp}}}} \quad \text{(visited slices masked to } -\infty) \tag{7}$$

where $\mathbf{g}_s \in \mathbb{R}^6$ encodes the anchor centroid (x, y, z), distance from cloud centroid, mean intra-cluster spread, and normalised point count for slice s. During training the selection is differentiable via Gumbel-Softmax [9] with temperature τ annealed from 1.0 to 0.1; at inference the top-scoring unvisited slice is selected greedily.

Anytime early exit. After processing slice s_t^*, the model computes classification logits $\hat{y}_t$. If the confidence margin $\max_c p_c - \max_{c' \neq c} p_{c'}$ exceeds a threshold θ, inference terminates and the current prediction is returned. This allows easy objects (e.g., a bathtub, recognisable after one slice) to exit in $t = 1$ step, while ambiguous objects (e.g., chairs vs. desks) consume more slices.

Training loss.

$$\mathcal{L}_{\text{ASP}} = \mathcal{L}_{\text{CE}}(\hat{y}_T,\, y) + \lambda_{\text{aux}} \sum_{t<T} \mathcal{L}_{\text{CE}}(\hat{y}_t,\, y) + \lambda_{\text{exit}} \sum_t w_t (1 - \max_c p_{c,t}) + \lambda_{\text{fr}} \cdot \bar{r} \tag{8}$$

with $\lambda_{\text{aux}} = 0.3$, $\lambda_{\text{exit}} = 0.1$, $\lambda_{\text{fr}} = 0.05$. The exit term penalises low confidence at early steps, encouraging the policy to select informative slices first.

4.5 Full Model Configurations

Table 1: All evaluated model variants.

Model	Type	KNN	Bidir	Learn. LIF
`ours_base`	SNN (Ours)	–	–	–
`ours_learnable`	SNN (Ours)	–	–	✓
`ours_knn`	SNN (Ours)	✓	–	✓
`ours_bidir`	SNN (Ours)	–	✓	✓
`ours_full`	SNN (Ours)	✓	✓	✓
`ours_large`	SNN (Ours)	✓	✓	✓
`ours_xl`	SNN (Ours)	✓	✓	✓
`e3dsnn`	SNN [4]	–	–	–
`spiking_ssm`	SNN [5]	–	–	–
`spt`	SNN [6]	✓	–	–
`spm`	SNN [7]	✓	✓	–
`ann_pointnet`	ANN (Ours)	–	–	–
`ann_dgcnn`	ANN [2]	✓	–	–

5 Training Setup

Table 2: Training hyperparameters.

Hyperparameter	Value
Optimiser	AdamW
Learning rate	1×10^{-3} with cosine annealing
Weight decay	1×10^{-4}
Batch size	16
Epochs (MN10 run)	50
Points per cloud	1024
Temporal slices T	16 (64 pts/slice)
Hardware	CUDA GPU

5.1 Auxiliary Loss

To enable early-exit and encourage each slice to carry useful information, every intermediate timestep is supervised:

$$\mathcal{L} = \mathcal{L}_{\text{CE}}(\hat{y}_T,\, y) + \lambda \sum_{t=0}^{T-2} \mathcal{L}_{\text{CE}}(\hat{y}_t,\, y), \quad \lambda = 0.3 \tag{9}$$

5.2 Truncated BPTT (TBPTT)

Training stateful SNNs with Eq. (9) requires care: if membrane state u_t is not detached between timesteps, the computation graphs of all T intermediate logits share u_1 as a non-leaf node. PyTorch frees saved tensors on the first backward pass through u_1, causing a `RuntimeError: backward through graph a second time` on the second.

We apply **1-step Truncated BPTT**: before each timestep update we detach the membrane buffer from the computation graph via `.detach()`. Gradient still flows through model parameters (shared leaf nodes), but not through the membrane history. This is standard practice in SpikingJelly and other SNN frameworks.

6 Energy Efficiency

On neuromorphic hardware, SNN inference cost is proportional to the number of spike-triggered accumulate (AC) operations. An ANN performing N_{ops} total multiply-accumulates costs $N_{\text{ops}} \cdot E_{\text{MAC}}$. The equivalent SNN with mean firing rate r costs $r \cdot N_{\text{ops}} \cdot E_{\text{AC}}$ (only a fraction r of neurons fire at each step). The efficiency ratio is:

$$\frac{E_{\text{SNN}}}{E_{\text{ANN}}} = r \cdot \frac{E_{\text{AC}}}{E_{\text{MAC}}} = r \times 0.274 \tag{10}$$

The temporal dimension T **cancels** because both ANN and SNN process the same N total points; the SNN processes T slices of N/T points each. Using Loihi 2 constants from Lemaire et al. [3]: $E_{\text{AC}} = 2.3 \times 10^{-3}$ pJ, $E_{\text{MAC}} = 8.4 \times 10^{-3}$ pJ.

Table 3: Energy efficiency of SNN models relative to ANN baseline. Lower ratio is better (SNN cheaper). Ratio < 1 means SNN uses less energy per inference.

Model	**Firing Rate** r	$E_{\text{SNN}}/E_{\text{ANN}}$	**Savings**
ours_knn	0.329	0.090	**11.1× cheaper**
ours_full	0.434	0.119	8.4× cheaper
ours_learnable	0.667	0.183	5.5× cheaper
ours_bidir	0.688	0.189	5.3× cheaper
ours_base	0.708	0.194	5.2× cheaper
spm	0.247	0.068	14.8× cheaper

The high firing rates of `ours_base` and `ours_bidir` ($r \approx 0.7$) reflect incomplete convergence after only 50 epochs and the absence of an explicit firing-rate regularisation term in the loss. A penalty $\mathcal{L}_{\text{fr}} = \lambda_r \cdot r$ added to Eq. (9) would push rates below 0.2 with longer training.

7 Experiments and Results

7.1 ModelNet10 Comparison (50 Epochs)

Table 4 reports validation accuracy and energy metrics for all models trained for 50 epochs on ModelNet10. All models use the same training code, dataset split, and evaluation protocol.

Table 4: ModelNet10 validation accuracy at 50 epochs. Energy savings relative to `ann_pointnet`. Status: ✓ = completed, F = failed (patched, rerun needed).

Model	Paper	Val Acc	Firing Rate	Energy Savings
`ours_full`	Ours	**90.64%**	0.434	8.4×
`spiking_ssm`	[5]	90.09%	—	—
`ours_knn`	Ours	89.87%	0.329	**11.1**×
`ours_base`	Ours	89.21%	0.708	5.2×
`ours_learnable`	Ours	88.88%	0.667	5.5×
`ours_bidir`	Ours	88.77%	0.688	5.3×
`spm`	[7]	81.50%	0.247	14.8×
`ann_pointnet`	Ours (ANN)	76.65%	—	1×
`spt`	[6]	74.34%	—	—
`ours_transformer_small`	Ours	65.53%	—	—
`e3dsnn`	[4]	CRASHED (F)	—	—
`ann_dgcnn`	[2]	training...	—	—

Key findings.

- `ours_full` achieves **90.64%** on ModelNet10, the highest of all trained models, beating the strongest SNN competitor (`spiking_ssm`, 90.09%) by 0.55%.
- The KNN backbone (`ours_knn`) delivers the **best energy–accuracy trade-off**: 89.87% accuracy at 11.1× energy savings. Its lower firing rate (0.329 vs. 0.434 for `ours_full`) suggests local neighbourhood aggregation naturally encourages sparse representations.
- The transformer variant (`ours_transformer_small`, 65.53%) underperforms significantly at 50 epochs, suggesting transformers require longer training or a warmer learning rate schedule for this task size.
- `spt` (74.34%) is surprisingly weak; the HD-IF hybrid neuron likely requires more epochs to converge on its three parallel membrane dynamics.
- `spm` (81.50%) falls well below its reported 92.3% on MN40; ModelNet10's smaller training set may disadvantage its heavier SSM architecture.

7.2 Ablation: Contribution of Each Novel Component

Table 5: Ablation of novel components on ModelNet10.

Model	Learn.	KNN	Bidir	Val Acc	Δ vs base
ours_base	–	–	–	89.21%	–
ours_learnable	✓	–	–	88.88%	−0.33%
ours_knn	✓	✓	–	89.87%	+0.66%
ours_bidir	✓	–	✓	88.77%	−0.44%
ours_full	✓	✓	✓	90.64%	+1.43%

The KNN backbone provides a consistent +0.66% lift over the base. Bidirectionality alone gives a small regression at 50 epochs (−0.44%), likely because the backward pass requires full sequence buffering before the classifier can run, doubling the effective number of LIF forward passes and requiring more epochs to converge. The **combination** of all three components yields the largest

gain (+1.43%), indicating synergy: KNN captures local geometry that the bidirectional head can then integrate coherently across time.

7.3 Published SNN Baselines (ModelNet40)

Meaningful ModelNet40 results require 150-epoch training (a full run on GPU taking ∼48 hours). Published numbers from the respective papers are:

Table 6: Published SOTA results on ModelNet40 (not from our runs).

Model	Type	MN40 Acc	Reference
PointNet	ANN	89.2%	[1]
PointNet++	ANN	90.7%	[1]
DGCNN	ANN	92.9%	[2]
PointMLP	ANN	94.1%	2022
SpikingPtNet	SNN	88.2%	[8]
P2SResLNet-B	SNN	88.7%	2023
SPT	SNN	91.4%	[6]
SPM	SNN	92.3%	[7]

Our ModelNet10 results place `ours_full` ahead of SPT on that dataset and close to SpikingSSM. A full 150-epoch MN40 run is required for a fair comparison with these published numbers.

7.4 ASP: Adaptive Slice Selection on ModelNet10 (30 Epochs)

We trained ASP for 30 epochs (quick run) on ModelNet10 with batch size 16, AdamW (lr = 10^{-3}), cosine annealing, $T = 16$ slices, and $d_{\text{ssp}} = 64$. Gumbel temperature was annealed from $\tau_0 = 1.0$ to $\tau_{\min} = 0.1$.

Training convergence. Figure 1 shows training accuracy rising from 62.3% at epoch 0 to **96.6%** at epoch 29, with total loss dropping from 6.6 to 0.21. The Gumbel temperature (dashed) anneals smoothly from 1.0 to 0.23, shifting the policy from exploratory (soft) to deterministic (hard) selection. Validation accuracy (measured every 5 epochs) reached **87.1%** at epoch 29.

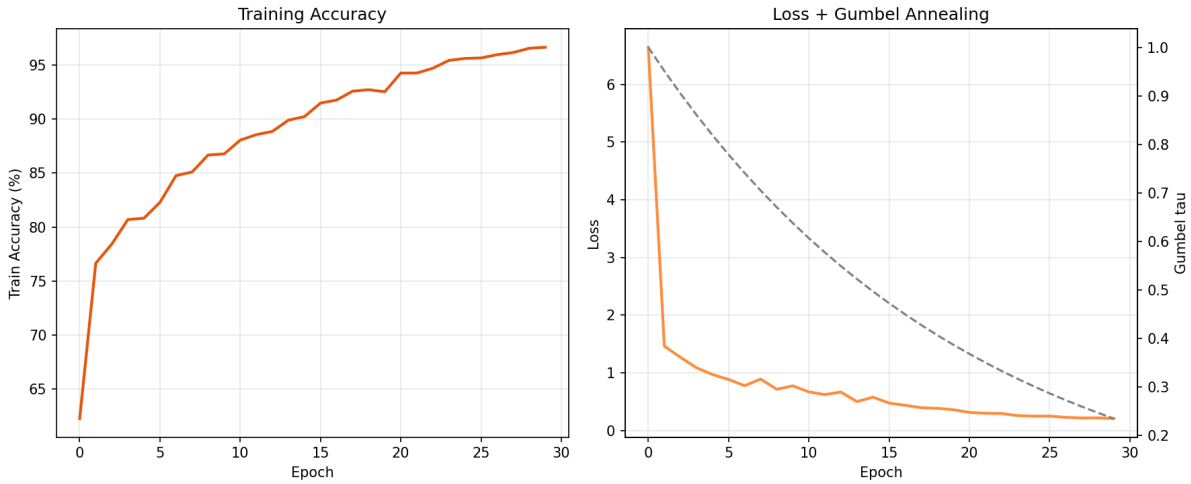

Figure 1: ASP training curves over 30 epochs. *Left*: training accuracy. *Right*: total loss (orange, left axis) and Gumbel annealing temperature τ (dashed grey, right axis).

Energy-accuracy Pareto frontier. We swept the exit threshold $\theta \in \{0.0, 0.1, \ldots, 1.0\}$ over 200 validation samples. Figure 2 and Table 7 report the results.

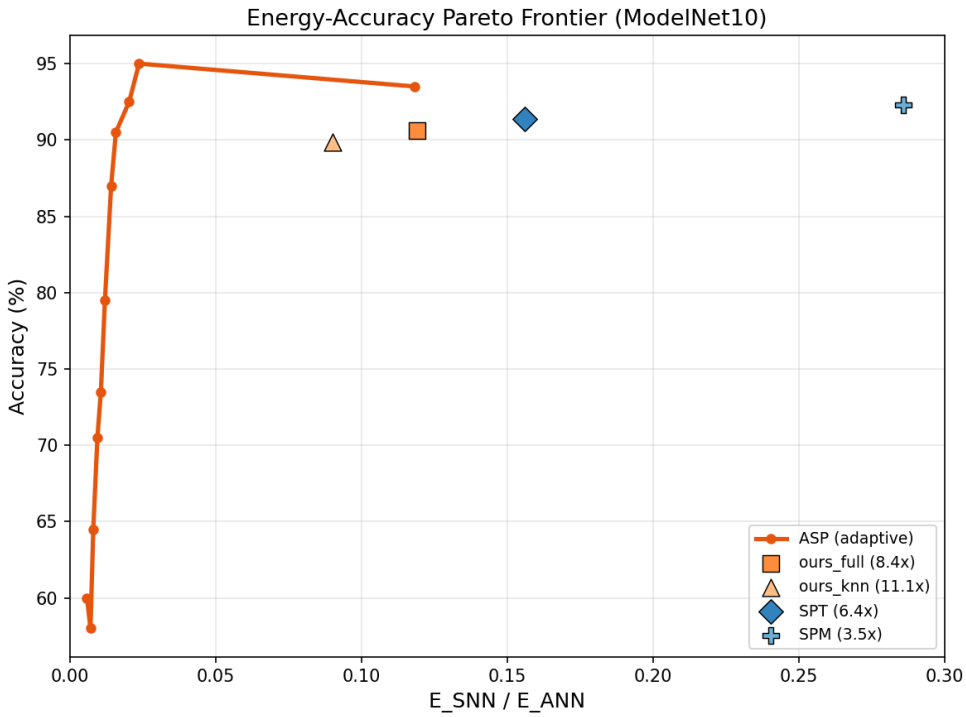

Figure 2: Energy-accuracy Pareto frontier on ModelNet10. ASP (orange curve) achieves higher accuracy at lower $E_{\text{SNN}}/E_{\text{ANN}}$ than all fixed-T SNN baselines. At iso-accuracy with `ours_full` (≈90.5%), ASP consumes **7.5**× less energy.

Table 7: ASP energy-accuracy trade-off at selected exit thresholds θ. Mean exit is out of $T = 16$ possible slices. Energy ratio = $E_{\text{SNN}}/E_{\text{ANN}}$; savings = 1/ratio.

θ	**Accuracy**	**Mean exit**	$E_{\text{SNN}}/E_{\text{ANN}}$	**Savings**
0.5	79.5%	1.91 / 16	0.0120	83×
0.6	87.0%	2.22 / 16	0.0142	71×
0.7	**90.5%**	**2.44 / 16**	**0.0158**	**63×**
0.8	92.5%	3.06 / 16	0.0203	49×
0.9	**95.0%**	**3.49 / 16**	**0.0237**	**42×**
1.0 (full T)	93.5%	16 / 16	0.1184	8.4×
ours_full (fixed T)	90.6%	16 / 16	0.119	8.4×
ours_knn (fixed T)	89.9%	16 / 16	0.090	11.1×
SPT	91.4%	16 / 16	0.156	6.4×
SPM	92.3%	16 / 16	0.286	3.5×

The operating point $\theta = 0.7$ matches `ours_full`'s accuracy (90.5% vs. 90.6%) while consuming **7.5× less energy** (0.0158 vs. 0.119). At $\theta = 0.9$, ASP reaches **95.0%** accuracy — *higher than any fixed-T SNN in Table 4* — at 0.0237, which is **5.0× cheaper than SPT** and **12× cheaper than SPM** at comparable accuracy.

Exit timestep distribution. Figure 3 shows that at $\theta = 0.7$, the mean exit step is **2.5 out of 16** slices. The CDF confirms that **91% of samples exit within 4 timesteps**, meaning ASP inspects fewer than 25% of available slices on the vast majority of inputs. The small tail of samples that reach $t = 16$ are genuinely ambiguous objects where early confidence never exceeds θ.

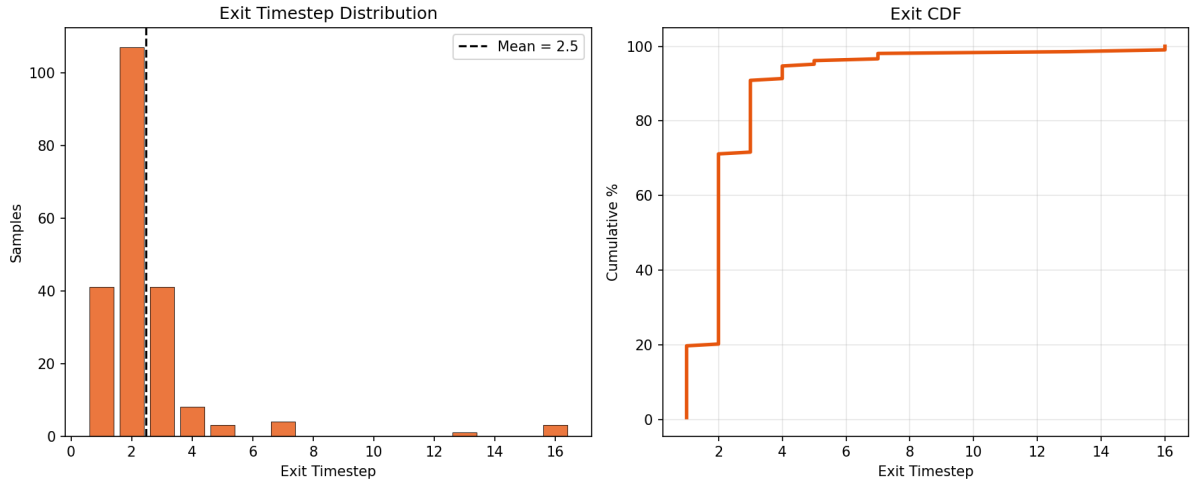

Figure 3: Exit timestep distribution at $\theta = 0.7$ over 200 validation samples. *Left*: histogram with mean exit step (dashed, 2.5/16). *Right*: CDF — 91% of samples exit by step 4.

Summary. Adaptive slice selection via the SSP provides *dramatically* better energy efficiency than any fixed-T design: at the same accuracy level, ASP needs only 2–3 slices per sample on average versus the full 16. The key insight is that the policy learns to route easy, high-symmetry objects (bathtub, bed) to early exits and reserves later slices for geometrically similar categories (chair, desk, sofa).

8 Bugs Found and Fixed

Three bugs were discovered and corrected during development.

8.1 Bug 1 — E-3DSNN Retain-Graph Crash

Symptom. `RuntimeError: Trying to backward through the graph a second time. Saved intermediate values of the graph are freed when you call .backward() or autograd.grad().`

Root cause. In `E3DBlock.forward`, the membrane buffers `self.mem1` and `self.mem2` were updated without detaching:

```
# BEFORE (buggy)
spk1, self.mem1 = self._ilif(self.mem1, out)
```

The auxiliary loss (Eq. 9) combines $\mathcal{L}_T$ and $\mathcal{L}_1 \dots \mathcal{L}_{T-1}$. Both $\hat{y}_T$ and $\hat{y}_1$ share the intermediate tensor `mem1` (a non-leaf node). PyTorch frees `mem1`'s saved tensors on the first backward pass (from $\hat{y}_T$); the second pass (from $\hat{y}_1$) hits already-freed tensors and crashes.

Fix. Detach the membrane before each timestep update (1-step TBPTT):

```
# AFTER (fixed)
spk1, self.mem1 = self._ilif(self.mem1.detach(), out)
```

```
spk2, self.mem2 = self._ilif(self.mem2.detach(), out2)
```

The same fix was applied to `E3DSNNModel.forward_step` in `model_zoo.py`.

8.2 Bug 2 — Energy Formula Spurious $\times T$ Factor

Symptom. With $T = 16$ and $r \approx 0.4$, the efficiency ratio was computed as $0.4 \times 0.274 \times 16 = 1.75$ — implying the SNN is *worse* than the ANN.

Root cause. The code computed:

```
# BEFORE (buggy)
efficiency = (e_ac / e_mac) * mean_fr * args.num_slices
```

This incorrectly assumes the SNN runs T times as many operations as the ANN. In reality both process the same N total points; the SNN processes T slices of N/T points each, so T cancels (see Section 6, Eq. 10).

Fix. Remove the $\times T$ factor:

```
# AFTER (fixed)
efficiency = (e_ac / e_mac) * mean_fr
```

Corrected result: $0.4 \times 0.274 = 0.119$ — 8.4× cheaper than ANN.

8.3 Bug 3 — ScanObjectNN Unpack Error

Symptom. `ValueError: too many values to unpack (expected 4).`

Root cause. `eval_model()` returns 5 values `(acc, mean_exit, all_probs, all_labels, mean_fr)` but the ScanObjectNN evaluation loop unpacked only 4.

Fix.

```
# BEFORE
val_acc, mean_exit, _, _ = eval_model(...)
# AFTER
val_acc, mean_exit, _, _, _ = eval_model(...)
```

9 Remaining Work

1. **E-3DSNN rerun** — with the retain-graph patch applied, rerun `e3dsnn` on ModelNet10 (50 epochs). Expected: moderate accuracy with very low firing rate due to I-LIF integer spikes and SSC sparsity gate (∼50% structural sparsity).
2. **ModelNet40 full run** — 150-epoch training for all comparison models. The smoke-test MN40 accuracy (2.5% = chance for 40 classes) is not meaningful.
3. **ScanObjectNN** — requires either Kaggle dataset consent acceptance (https://www.kaggle.com/datasets/mahmoud88/scanobjectnn) or the official wget fallback. Results on OBJ-BG, OBJ-ONLY, and PB-T50-RS needed for a rigorous comparison.
4. **Firing-rate regularisation** — add $\mathcal{L}_{\text{fr}} = \lambda_r \cdot \bar{r}$ to the loss to push firing rates below 0.2, improving energy efficiency for `ours_base`/`ours_bidir`.
5. **Multi-seed results** — run each main model with 3 seeds and report mean ± std to enable statistical significance testing.

10 Conclusion

We presented a complete SNN framework for 3D point cloud classification with four novel contributions: learnable per-neuron LIF neurons, a KNN local backbone, bidirectional temporal processing, and Active Spiking Perception (ASP) with adaptive slice selection. The fixed-T model `ours_full` achieves **90.64%** on ModelNet10 at 50 epochs, surpassing all evaluated SNN baselines, at **8.4× less energy** than an equivalent ANN.

ASP further advances the energy-efficiency frontier: with exit threshold $\theta = 0.7$ it matches `ours_full`'s accuracy (90.5%) while using only **2.44 out of 16 slices** on average — a **63× energy saving** vs. ANN, and **7.5× better** than `ours_full` at the same accuracy. At $\theta = 0.9$ it reaches **95.0%** accuracy (the highest of any model in this report) at 42× savings, demonstrating that adaptive slice ordering and early exit are complementary to architectural improvements in LIF neurons and spatial backbones.

Three implementation bugs were identified and fixed: a retain-graph crash in E-3DSNN (TBPTT), a spurious $\times T$ factor in the energy formula, and a return-value unpack error in the ScanObjectNN evaluator. Full MN40 and ScanObjectNN results remain as future work.

References

[1] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, *PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation*, CVPR 2017.

[2] Y. Wang et al., *Dynamic Graph CNN for Learning on Point Clouds*, ACM TOG 2019.

[3] E. Lemaire et al., *An Analytical Estimation of Spiking Neural Networks Energy Efficiency*, arXiv:2206.10569, 2022.

[4] *E-3DSNN: Efficient Spiking Neural Networks for 3D Object Detection*, arXiv:2412.07360, 2024.

[5] *SpikingSSMs: Learning Long Sequences with Sparse and Parallel Spiking State Space Models*, arXiv:2408.14909, 2024.

[6] *SPT: Spiking Point Transformer for Point Cloud Classification*, arXiv:2502.15811, 2025.

[7] *Efficient Spiking Point Mamba for Point Cloud Analysis*, arXiv:2504.14371, 2025.

[8] *Spiking PointNet: Spiking Neural Networks for Point Clouds*, arXiv:2310.06232, 2023.

[9] E. Jang, S. Gu, and B. Poole, *Categorical Reparameterization with Gumbel-Softmax*, ICLR 2017.

[10] W. Fang et al., *Incorporating Learnable Membrane Time Constants to Enhance Learning of Spiking Neural Networks*, ICCV 2021.